# The Provenance Forest

Poker's generalisation step without enumerating refutations

Oruži Zarathustra Goertzel

7 November 2026

**Abstract**

Poker generalises an example by collecting, for every refutation of the example within the clause limit, the set of metasubstitutions the refutation uses. Vanilla enumerates these refutations one at a time, and their number grows exponentially with the clause limit. The provenance forest computes the same sets by dynamic programming: it solves each first-order goal once for each exact size, and builds the sets of a refutation from those of its parts. On Poker's $a^n b^n$ experiment it returns exactly Poker's sub-hypotheses in every case compared, for example the same 18 at $a^4b^4$ with clause limit 7 in 3.4 ms where Poker takes 218 s.

## 1 The problem

Poker's generalisation step takes an example $e$, background knowledge $B$, a set of metarules, a signature (the target predicate and the invented predicates) and a clause limit $K$. Each refutation of $e$ resolves goals either against $B$ or against an instance of a metarule; each such instance is a *metasubstitution* $\sigma$, the metarule with its second-order variables bound. The *support* of a refutation is the set of metasubstitutions it uses. Poker's sub-hypotheses are the supports of the refutations of $e$ with at most $K$ steps.

Vanilla finds them by SLD-resolution with the metasubstitutions found so far as an argument of every call. Distinct refutations share most of their sub-refutations, but an untabled prover derives each shared part again in every refutation that contains it. Tabling the meta-interpreter does not recover the sharing: two calls on the same goal with different stores are different calls. On $a^4b^4$ at clause limit 7 Poker spends 5.13 billion inferences to produce 18 sets.

## 2 The decomposition

A refutation is a tree. Its root is a metasubstitution $\sigma$ whose head matches the goal; its children are refutations of the body literals of $\sigma$; its leaves are background facts. Size and support are both compositional:

$$\mathrm{size}(t) = 1 + \sum_i \mathrm{size}(t_i), \qquad \mathrm{supp}(t) = \{\sigma\} \cup \bigcup_i \mathrm{supp}(t_i),$$

with size 0 and support $\emptyset$ at a background leaf. The size counts steps with multiplicity, while the support is a set, as in Poker's store. Neither depends on the rest of the tree, so the family of supports of a goal's refutations of size exactly $n$,

$$F_n(g) = \{\, \mathrm{supp}(t) : t \text{ a refutation of } g,\ \mathrm{size}(t) = n \,\},$$

satisfies

$$F_0(g) = \{\emptyset\} \text{ if } g \in B, \text{ and } \emptyset \text{ otherwise,}$$
$$F_{n+1}(g) = \bigcup_{\sigma : g \leftarrow g_1,\ldots,g_m} \; \bigcup_{n_1+\cdots+n_m=n} \{\, \{\sigma\} \cup S_1 \cup \cdots \cup S_m : S_i \in F_{n_i}(g_i) \,\},$$

and the generalisation of $e$ at clause limit $K$ is $\bigcup_{n=1}^{K} F_n(e)$. Each $F_n(g)$ is computed once and shared by every refutation that contains $g$ at size $n$.

**Theorem 1** (the recurrence characterises the supports)**.** *For a ground derivation forest (goals, background goals, and edges $g \leftarrow \sigma, [g_1, \ldots, g_m]$), the family $F$ of supports of derivation trees satisfies the recurrences above, and every family satisfying them equals $F$.*

*Proof sketch.* That $F$ satisfies the recurrences is the decomposition of a tree at its root. For uniqueness, let $G$ satisfy them and show $G_n(g) = F_n(g)$ for all $g$ by strong induction on $n$: at $n = 0$ both are given by $B$; at $n + 1$ both sides apply the same recurrence to families of sizes $n_i \leq n$, which agree by induction. □

**Lemma 2** (pruning loses nothing)**.** *In the recurrence, a splitting $n_1 + \cdots + n_m = n$ contributes a set only if each body literal $g_i$ has a derivation of size $n_i$. Hence restricting the splittings to those in which every literal is known to be derivable at its share keeps every set of $F_{n+1}(g)$.*

Both statements are machine-checked in Lean 4 with Mathlib, using no axioms beyond Lean's standard ones, together with positive and negative examples.

## 3 From ground forests to MIL goals

In meta-interpretive learning the forest is not given in advance. A goal is $P(x, Y)$ with $x$ known; the body of Chain, $Q(X, Z), R(Z, Y)$, introduces an existential $Z$ that is known only once $Q(x, Z)$ is solved. The forest therefore works in two passes over memo tables keyed by goal and size.

**Pass 1: endpoints.** *reach*$(P, x, n)$ is the set of endpoints $y$ such that $P(x, y)$ has a refutation of size exactly $n$. A body is solved left to right: the size $n - 1$ is split among the literals, and the endpoints of each literal at its share instantiate the next literal's input. A literal of size 0 is answered by $B$; a literal of positive size is a target or invented predicate, answered by *reach* itself.

**Pass 2: supports.** *fam*$(P, x, y, n)$ is $F_n(P(x, y))$, built by the recurrence, but only over the splittings pass 1 found feasible. By the pruning lemma this loses nothing.

**Invented predicates.** Each memo key also carries which invented predicates are available: the $i$-th invented predicate may head a clause only once the $(i-1)$-th is available. This names inventions canonically, so that supports differing only by a renaming of invented predicates are not produced twice. A goal of the ground forest is then a goal together with this state, and the theorem applies unchanged.

**Metarule constraints.** Poker's constraints, such as $P \neq Q$ in Chain, are tests on a single metasubstitution. They are applied as soon as a metasubstitution is built, comparing symbols with `==` so that a symbol not yet bound passes. A constraint on one metasubstitution removes edges from the forest and does not interact with the decomposition.

## 4 Cost

The memo tables hold one entry per goal, size and invention state. Pass 1 costs, per entry, one step for each metarule, choice of symbol, splitting of the size and intermediate endpoint. Pass 2 builds only sets that are parts of sub-hypotheses: every set it builds is contained in one of the output's sets. Poker's cost grows with the number of refutations instead, and of the partial ones that fail, a number that grows exponentially with $K$ even when the number of distinct supports does not: at $a^4b^4$, limit 7, there are 18 supports.

Table 1 reports Poker's `generalise/3` and the forest on the same problems, in SWI-Prolog 10.1.9. The sets of sub-hypotheses are identical in every row. At limits too small to fit any refutation both return none (Poker's `generalise/3` fails, which the comparison reads as none).

| Task | Sub-hypotheses | Poker | | Forest | |
|---|---|---|---|---|---|
| | | time | inferences | time | inferences |
| $a^1b^1$, $K = 1$ | 1 | 0.9 ms | 5,956 | 0.2 ms | 1,804 |
| $a^2b^2$, $K = 3$ | 4 | 9.3 ms | 199,702 | 0.4 ms | 6,399 |
| $a^2b^2$, $K = 5$ | 1,449 | 1.68 s | 35,883,880 | 6.9 ms | 79,931 |
| $a^3b^3$, $K = 5$ | 10 | 1.20 s | 29,253,930 | 1.1 ms | 23,176 |
| $a^3b^3$, $K = 7$ | 9,941 | 219 s | 5,348,394,103 | 124 ms | 711,019 |
| $a^4b^4$, $K = 7$ | 18 | 218 s | 5,134,352,712 | 3.4 ms | 65,625 |

Table 1: Poker against the forest on Poker's $a^nb^n$ experiment (single runs; the inference counts are exact).

On its own the forest takes 7.1 ms on $a^5b^5$ at $K = 9$ and 32 ms on $a^7b^7$ at $K = 13$.

## 5 Scope

- The decomposition assumes that a refutation resolves goals against background knowledge and metarules only, not against clauses of the hypothesis being built, so that every step instantiates a metarule and the clause limit bounds the steps. This is the setting of the $a^nb^n$ experiment: there Poker finds no refutation of $a^3b^3$ at limit 4, although a grammar of three clauses derives it in five steps. When hypothesis clauses can be reused, size is no longer additive; extending the forest to that setting is open.
- The forest replaces the generalisation step only. Poker's other stages are unchanged.
- Tasks are currently given in the forest's own encoding; reading Vanilla experiment files directly is the next step.

## 6 Running it

The forest is in `lib/forest`; tasks and scripts are in `data/forest_examples`. From Vanilla's root, with Poker selected as the learner in `configuration.pl`,

```
swipl data/forest_examples/poker_comparison.pl N K
```

runs both learners on $a^Nb^N$ at clause limit $K$ and compares their sub-hypotheses as sets of sets. The forest was first written in MeTTa, where each memo table is a space of shared immutable

terms indexed by their arguments; the files here are its port to SWI-Prolog, with the memo tables as asserted facts keyed by term hash or as tabled predicates.